

CITS3005 Knowledge Representation Homework 2

Student: Dean McCullough

Student No: 24689126

Part 1: Description.

The knowledge graph represents the fictional world of *Made in Abyss*. The story centres on Orth, a city built around *The Abyss*, a giant hole. Orth became prosperous because delvers could find artifacts from the Abyss and sell them overseas, motivating the profession. Descending into the Abysses' layers becomes increasingly dangerous the deeper you delve. This is due to hostile creatures, dangerous environments, and the *Curse of the Abyss*, which causes harmful effects when a person ascends from deeper layers. The deeper layers are also less explored and less understood, making incomplete knowledge an important feature of this world.

The graph uses an open world assumption. If a fact is not represented in the graph, this does not mean that the fact is false. I think this reflects the mystery surrounding the abyss with the graph helping to categorise and understand the mysterious and unknown entities of the abyss.

Entity Types

- Entity: The superclass for all named things represented in the knowledge graph.
- Place: A broad geographical area or region in the world.
- District: A subsection of a place. A subclass of Place
- Location: A specific location situated within a place. Subclass of Place
- Object: A non-living physical thing that exists in the world.
- Item: A known item in the world. E.g. Whistle. Items are a subclass of objects.
- Artifact: A special object from the Abyss. Artifacts may have owners, effects, grades, and sources. Artifacts are a subclass of items.
- Living_Thing: The superclass for living things in the world.
- Beast: A non-humanoid living creature. E.g. *A creature of the abyss*.
- Human: A human or human-like living being.
- Narehate: Beings who have survived the 6th layers curse.

Meta Types:

I have also included a few meta data classes for query purposes.

- Danger level: A rating indicating how dangerous a beast is.
- Whistle Rank: A rating indicating the ability of a diver, and layer conditions.
- Artifact Grade: A rate indicating how powerful a known relic is.

Relations:

- hasDescription: Indicates an entity has a description associate.
- hasEffect: Describes an effect produced by an object, place or living thing.

Location Relations:

- contains: Connects a broader place to a place or location situated within it.
- locatedIn: Connects an entity to the place or location where it is found.

- hasWhistleRequirement: Connects a layers entry conditions to whistle rank.
- hasDepth: Has a description of the depth an entity exists
- minDepth: Relation for resolving filter queries
- maxDepth: Relation for resolving filter queries,

Object Relations:

- ownedBy: Connects an object or artifact to its current or former owner.
- sourcedFrom: Connects an artifact to the place from which it originated or was recovered.
- hasGrade: Connects an artifact to its grade.

Living Entity Relations:

- lastKnownLocation: Connects a human to their last known location.
- hasGender: Gives an entity's gender as a literal value.
- hasStatus: A character status, such as alive, deceased, or unknown.
- hasSpecies: Gives a character a species as a literal.
- hasOccupation: Connects a person to an occupation as a literal.
- hasWhistleRank: Gives a delver's whistle rank as a literal value.
- livesIn: Connects a creature to a location it inhabits.
- hasDangerLevel: Gives a danger level associated with an entity as a literal value.

2. Describe a set of useful queries that your knowledge graph could be used to answer.

Queries can fall under a set of related queries like listing, filtering or aggregation. Some are mainly useful internally to be used for utility and extend functionality.

List:

- List entities: List all entities
- List entity types: List all classes
- List property values: List property values (Utility mostly)
- List relations: List available relations
- List danger levels: Lists available danger levels
- List whistle rank: List available whistles
- List layers: List the layers of the abyss

Filter:

- Filter by name: Accept a string and return entities with labels matching.
- Filter by danger: Accept a danger level input and return all monsters with the same danger level.
- Filter by property value: Accept a predicate and relation and find corresponding entities.
- Filter by relation: Accept relation choice and find all subject to object relations .
- Filter by depth: Accept an integer value and filter for entities in a range.
- Filter by entity type: Accept an entity type and return entities of that type.

Aggregation:

- Aggregate entities: Count number of entities
- Aggregate properties: Count number entities tied to a relation

Utility:

Functions to support more advanced querying. This can be listing or filters for more complex queries to be worked on.

- Get label: Gets the label associated with an entity
- Get parents: Get the superclasses recursively of an entity.

Part 2:

See:

“made_in_abyss_graph.rdf”

“made_in_abyss_queries.py”

I think a lot of improvements can be made by designing a tiered system for querying allowing more complex functionality without individual complex functions. Designing this will be useful for the project and give availability to extend our graph without a major headache. We also should be mindful of how our graph represents data as that is a massive limitation, some query ideas fell flat due to this issue and would require redesign to support it.

Part 3:

See:

“made_in_abyss_problog.pl”

I’ll be honest and say I had a tough time designing anything for this, after attempting a few times, I couldn’t find a meaningful interpretation through problog. We are modelling uncertainty through a rather abstracted lens, hence the difficulty. I ended on a survival rate which I kind of produced through some math principles but it works with some limitations unfortunately.